



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 2: Componentes de una computadora y procesos

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 20 de agosto de 2026

Índice de figuras

1.1.	Componentes principales de una computadora y su comunicación mediante el System Bus.	4
1.2.	Basic Instruction Cycle.	6
1.3.	Formato de instrucciones y registers de una máquina hipotética.	7
2.1.	Ejercicio 2 ejecutado en WSL en un procesador x86	9
2.2.	Ejercicio 2 ejecutado en macOS en un procesador ARM	9
3.1.	Ejercicio ejecutado en JAVA	12

Lista de abreviaturas y símbolos

- OS** *Operating System*: sistema operativo que administra los recursos de hardware.
- CPU** *Central Processing Unit*: procesador que controla la computadora.
- RAM** *Random Access Memory*: memoria principal que mantiene temporalmente instrucciones y datos.
- I/O** *Input/Output*: entrada y salida de datos.
- PC** *Program Counter*: contiene la dirección de la siguiente instrucción.
- IR** *Instruction Register*: contiene la instrucción que está siendo ejecutada.
- MAR** *Memory Address Register*: especifica la dirección de memoria que será leída o escrita.
- MBR** *Memory Buffer Register*: contiene los datos que se escriben o leen de la memoria.
- I/O AR** *Input/Output Address Register*: especifica el dispositivo de I/O.
- I/O BR** *Input/Output Buffer Register*: intercambia datos con un I/O module.
- AC** *Accumulator*: register utilizado como almacenamiento temporal.
- CISC** *Complex Instruction Set Computer*: arquitectura de procesador con un conjunto de instrucciones complejas e integradas.
- RISC** *Reduced Instruction Set Computer*: arquitectura de procesador simplificada orientada a la ejecución rápida de instrucciones básicas.
- x86** Arquitectura de juego de instrucciones CISC desarrollada originalmente por Intel para procesadores de 32 y 64 bits (x86_64).
- ARM** *Advanced RISC Machine*: arquitectura de procesamiento basada en diseño RISC de alto rendimiento y bajo consumo energético.
- PID** *Process Identifier*: identificador único asignado por el kernel a un proceso en ejecución.
- PPID** *Parent Process Identifier*: identificador de proceso correspondiente al proceso padre que generó la ejecución actual.

UID *User Identifier*: número entero que identifica de forma persistente al usuario propietario del proceso.

EUID *Effective User Identifier*: identificador de usuario efectivo que el sistema evalúa para otorgar permisos y accesos en tiempo de ejecución.

1. Funcionamiento de una Computadora

Explica lo más claro posible el funcionamiento de cada componente de una computadora con base en la lectura del punto anterior, centrándose en el tema 1.3 instruction execution.

1.1 Componentes de una computadora e Instruction Execution

En general, una computadora tiene cuatro componentes principales: el procesador procesa los datos, la *main memory* mantiene las instrucciones y los datos, los *I/O modules* los mueven entre la computadora y los dispositivos externos, y el *System Bus* permite la comunicación entre el CPU, la main memory y los I/O modules.

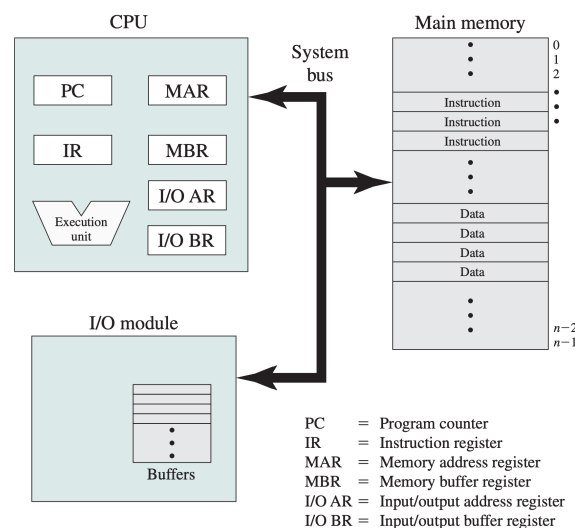


Figura 1.1: Componentes principales de una computadora y su comunicación mediante el System Bus.

1.1.1 CPU y registers

El *Central Processing Unit* (CPU) es el procesador que controla la operación de la computadora. Dentro del CPU existen los *registers*, que guardan instrucciones, direcciones o datos que se necesitan en ese momento.

El *Program Counter* (PC) contiene la dirección de la siguiente instrucción de la que se realizará el *fetch*. El *Instruction Register* (IR) contiene la instrucción que se está ejecutando, mientras que la *Execution Unit* realiza la operación indicada por esa instrucción.

El *Memory Address Register* (MAR) especifica qué dirección de memoria será la siguiente en ser leída o escrita. El *Memory Buffer Register* (MBR) contiene los datos que se escribirán en la memoria o recibe los datos que se leen de ella.

Para las operaciones de I/O, el *I/O Address Register* (I/O AR) especifica con cuál dispositivo se realizará la operación, mientras que el *I/O Buffer Register* (I/O BR) se utiliza para intercambiar datos entre el procesador y un I/O module.

1.1.2 Main Memory

La *main memory*, también llamada *Random Access Memory* (RAM), guarda temporalmente las instrucciones y los datos que necesita el procesador. Es una memoria *volatil*, por lo que su contenido desaparece cuando la computadora se apaga. En cambio, la memoria secundaria, como un disco o SSD, conserva los datos.

También existe una *memory hierarchy*. Los registers están dentro del CPU y mantienen los valores que se necesitan inmediatamente. Después se encuentran las memorias caché L1, L2 y L3, que normalmente están integradas en el chip del procesador. Luego está la RAM y finalmente la memoria secundaria, como un SSD.

1.1.3 I/O Modules

Los I/O devices son los dispositivos físicos, como un disco, un SSD, un teclado, un mouse o unos audífonos. El I/O module es el componente interme-

diario que permite que esos dispositivos se comuniquen con el procesador y la main memory. Por ejemplo, el teclado y el mouse entregan datos de entrada, los audífonos reciben datos de salida y un disco o SSD puede leer y escribir datos.

El *buffer* es un espacio temporal que mantiene temporalmente bloques de datos y *amortigua* la diferencia de velocidad entre dispositivos, mientras esperan ser enviados, recibidos o procesados. Esto evita que cada componente tenga que esperar y transferir los datos uno por uno.

1.1.4 Programa e Instruction Cycle

Programa: es un *set de instrucciones* guardadas en memoria que el procesador obtiene mediante *fetch* y después ejecuta.

El procesamiento de una instrucción tiene dos etapas:

1. **Fetch stage:** el procesador realiza el fetch de la siguiente instrucción desde la memoria. El PC contiene su dirección y la instrucción obtenida se coloca en el IR.
2. **Execute stage:** el procesador interpreta la instrucción y la Execution Unit realiza la operación indicada.

Al procesamiento completo de una instrucción se le llama *Instruction Cycle*. El procesador repite el fetch stage y el execute stage porque un programa contiene varias instrucciones que deben ejecutarse.

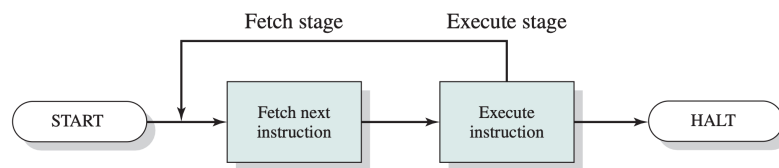


Figure 1.2 Basic Instruction Cycle

Figura 1.2: Basic Instruction Cycle.

La Figura 1.2 resume este proceso. Después de cada fetch, el PC aumenta para apuntar a la siguiente dirección de memoria. El proceso termina cuando se encuentra una instrucción de *HALT*, (o cuando se apaga el procesador o cuando ocurre un error).

1.1.5 Formato de una instrucción

En la Figura 1.3, cada instrucción tiene 16 bits. Los primeros 4 bits (que esta formado por una letra en formato hexadecimal) forman el *opcode*, que indica la operación que realizará el procesador, y los 12 bits restantes forman el *address*, que indica una dirección de memoria.



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
 Instruction register (IR) = Instruction being executed
 Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
 0010 = Store AC to memory
 0101 = Add to AC from memory

(d) Partial list of opcodes

Figura 1.3: Formato de instrucciones y registers de una máquina hipotética.

1.1.6 Instrucciones

La ejecución de una instrucción puede producir cuatro tipos generales de acciones:

- **Processor-memory:** se transfieren datos entre el procesador y la memoria.
- **Processor-I/O:** se transfieren datos entre el procesador y un I/O device.
- **Data processing:** el procesador realiza una operación.
- **Control:** una instrucción modifica la secuencia de ejecución del PC.

1.1.7 Ejecución de un programa mediante los componentes de la computadora

Antes de ejecutar un programa, este se encuentra almacenado en la memoria secundaria. El OS carga en la main memory las instrucciones y los datos necesarios desde el dispositivo de almacenamiento, a través de un *I/O module* y sus buffers, utilizando el *System Bus*.

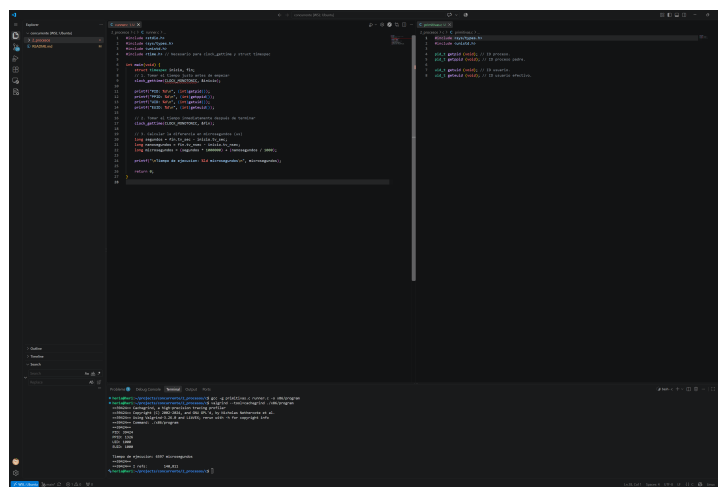
Una vez que las instrucciones están cargadas, el CPU comienza el *Instruction Cycle*. Este proceso ocurre de la siguiente manera:

1. El PC contiene la dirección de la siguiente instrucción de la que se realizará el fetch.
2. La dirección se coloca en el MAR, que especifica la ubicación de memoria que debe leerse.
3. La main memory entrega la instrucción. Esta pasa por el MBR y después se coloca en el IR.
4. El PC normalmente aumenta para apuntar a la siguiente instrucción guardada en memoria.
5. El CPU interpreta la instrucción que se encuentra en el IR con el *opcode*, que indica qué operación debe realizarse, y un *address*, que señala una dirección de memoria.
6. La *Execution Unit* ejecuta la operación indicada por el opcode.
7. El resultado puede mantenerse temporalmente en un register, guardarse en la main memory o enviarse a un I/O device.
8. El CPU vuelve a realizar el fetch de la siguiente instrucción y repite el Instruction Cycle.

La memoria secundaria conserva el programa, los I/O modules permiten transferirlo, los buffers mantienen temporalmente los datos, el System Bus comunica los componentes, la RAM mantiene las instrucciones y los datos que se están utilizando, y el CPU realiza repetidamente los fetch stages y execute stages hasta encontrar una instrucción de *HALT*.

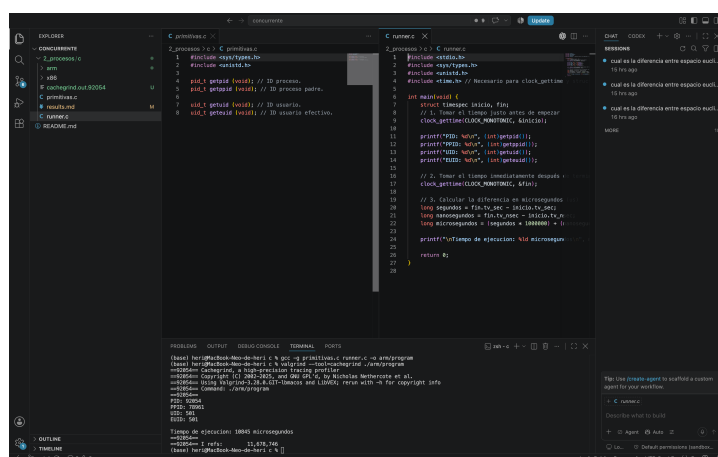
2. Instrucciones e Identificadores en C

Ejecuta el siguiente código fuente en C en una máquina virtual de Linux y agrega la captura de pantalla de la ejecución.



The screenshot shows a code editor with a C program that calculates the difference in execution time between two functions. The program includes `<stdio.h>`, `<time.h>`, and `<unistd.h>`. It defines two functions, `funcion_1` and `funcion_2`, which perform a loop of 10 million iterations. The main function calls `clock_gettime(CLOCK_REALTIME, &ts1)` before `funcion_1`, `clock_gettime(CLOCK_REALTIME, &ts2)` before `funcion_2`, and `clock_gettime(CLOCK_REALTIME, &ts3)` after `funcion_2`. It then calculates the difference in time between `ts1` and `ts2`, and between `ts2` and `ts3`, and prints the results. The output shows that `funcion_1` took approximately 1.000000 seconds and `funcion_2` took approximately 1.000000 seconds.

Figura 2.1: Ejercicio 2 ejecutado en WSL en un procesador x86



The screenshot shows a code editor with the same C program as in Figure 2.1. The program is executed on a macOS machine with an ARM processor. The output shows that `funcion_1` took approximately 1.000000 seconds and `funcion_2` took approximately 1.000000 seconds. The program also prints the total time of execution, which is 1.000000 seconds.

Figura 2.2: Ejercicio 2 ejecutado en macOS en un procesador ARM

2.1 Análisis Comparativo de Instrucciones e Identificadores: x86_64 (WSL) vs. ARM64 (macOS)

A partir de los reportes generados por la herramienta Cachegrind de Valgrind, se analiza la ejecución del programa C en dos arquitecturas de hardware distintas: un procesador Intel Core i5-13500H (x86_64) corriendo bajo WSL (Linux) y un Apple A18 Pro (ARM64) corriendo en macOS nativo.

2.1.1 Comparativa de Identificadores POSIX y Entornos de Ejecución

Los metadatos extraídos por el programa reflejan las diferencias estructurales entre la capa de compatibilidad de Windows (WSL) y el sistema operativo macOS:

- **Gestión de Usuarios (UID / EUID):** En el entorno WSL (Linux), el primer usuario no administrativo recibe por estándar el valor 1000 (**herie**). En macOS, la convención del sistema asigna el valor 501 a la cuenta del usuario principal (**heri**). En ambos escenarios, al no haber elevación de privilegios, UID = EUID.
- **Jerarquía de Procesos (PID / PPID):** En WSL, la terminal interactiva opera con el PPID = 1326, creando el proceso hijo con PID = 39424. En macOS, la terminal (zsh) bajo el proceso PPID = 78961 engendra al ejecutable asignándole el PID = 92054.

2.1.2 Mapeo de Instrucciones (I refs) y Ecuación de Rendimiento de la CPU

La métrica **I refs** (*Instruction references*) representa el total de instrucciones procesadas. El tiempo total de ejecución depende de las características microarquitectónicas y a la frecuencia de reloj del procesador, aproximadamente:

$$T = \frac{IC \times CPI}{f}$$

Donde:

- T : Tiempo de ejecución del procesador.
- IC (*Instruction Count*): Número total de instrucciones ejecutadas (**I refs**).
- CPI (*Cycles Per Instruction*): Promedio de ciclos de reloj requeridos para ejecutar una instrucción.
- f : Frecuencia de reloj del procesador (Hertz).

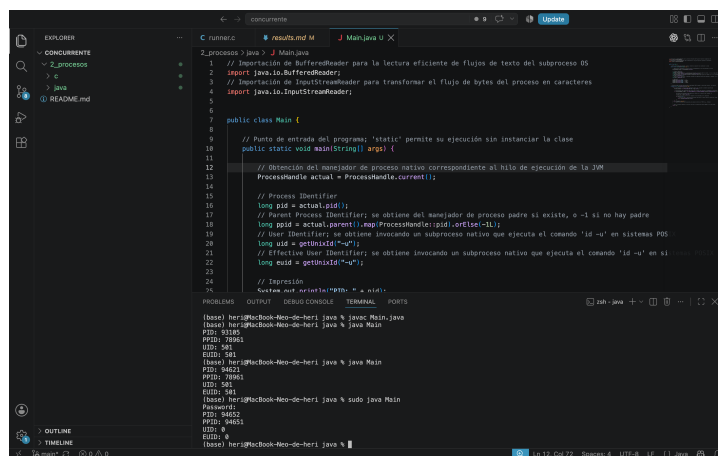
2.1.3 Análisis Microarquitectónico: Intel i5-13500H vs. Apple A18 Pro

Las discrepancias en los registros se explican por el diseño de cada chip:

- **Intel Core i5-13500H (x86_64 / CISC)**: Este chip opera con una frecuencia máxima de 4.7 GHz. CISC (*Complex Instruction Set Computer*) permite codificar operaciones densas (como accesos a memoria y cálculos aritméticos) en una sola instrucción. Esto reduce el IC (140,811 I refs), pero a costa de un CPI más alto por instrucción.
- **Apple A18 Pro (ARM64 / RISC)**: Opera con una frecuencia máxima de 4.04 GHz en sus núcleos de rendimiento. RISC (*Reduced Instruction Set Computer*) usa operaciones sencillas. Esto genera un aumento de IC (11,678,746 I refs), compensada por un CPI bajo (cercano a 1 por ciclo).

3. Instrucciones e Identificadores en JAVA

El código fuente anterior como lo implementaría en JAVA, agrega tu código en JAVA y la captura de pantalla de su ejecución.



```
1 // Importación de BufferedReader para la lectura eficiente de flujos de texto del subproceso OS
2 import java.io.BufferedReader;
3 // Importación de InputStreamReader para transformar el flujo de bytes del proceso en caracteres
4 import java.io.InputStreamReader;
5
6 public class Main {
7
8     // Punto de entrada del programa; 'static' permite su ejecución sin instanciar la clase
9     public static void main(String[] args) {
10
11         // Obtención del manejador de proceso nativo correspondiente al hilo de ejecución de la JVM
12         ProcessHandler actual = ProcessHandler.current();
13
14         // Process Identifier
15         long pid = actual.pid();
16         // Parent Process Identifier: se obtiene del manejador de proceso padre si existe, o -1 si no hay padre
17         long ppid = actual.parent().map(ProcessHandler::pid).orElse(-1L);
18         // User Identifier: se obtiene invocando un subproceso nativo que ejecuta el comando 'id -u' en sistemas POSIX
19         long uid = getuid();
20         // Effective User Identifier: se obtiene invocando un subproceso nativo que ejecuta el comando 'id -u' en si
21         long euid = geteuid();
22
23         // Impresión
24         System.out.println("PID: " + pid);
25     }
26 }
27
28 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
29 (base) her@MacBook-Mac-de-heri: java % java Main.java
30 PID: 93105
31 PPID: 78951
32 UID: 501
33 EUID: 501
34 (base) her@MacBook-Mac-de-heri: java % java Main
35 PID: 94621
36 PPID: 78951
37 UID: 501
38 EUID: 501
39 (base) her@MacBook-Mac-de-heri: java % sudo java Main
40 Password:
41 PID: 94633
42 PPID: 94621
43 UID: 0
44 EUID: 0
45 (base) her@MacBook-Mac-de-heri: java %
```

Figura 3.1: Ejercicio ejecutado en JAVA

3.1 Análisis de Procesos

Al ejecutar el programa desde la terminal de VS Code, el sistema operativo devuelve diferentes valores para los identificadores de proceso e identidad según el tipo de ejecución.

3.1.1 Cambio de PID y Permanencia de PPID

En las dos primeras ejecuciones (java Main), se observan los siguientes comportamientos:

- **PID (*Process ID*):** Cambia en cada ejecución (93105 en la primera y 94621 en la segunda). Cada vez que se corre un programa, el sistema

operativo le asigna un número de identificación único y temporal que se destruye al finalizar.

- **PPID (*Parent Process ID*)**: Se mantiene constante en 78961. Esto ocurre porque ambas ejecuciones fueron lanzadas desde la misma ventana de la terminal (zsh). La terminal es el proceso "padre" que abre al programa "hijo". No se muestra en la figura, pero al abrir otra terminal se obtuvo PPID: 94952

3.1.2 Cambio de Privilegios con sudo (UID y EUID)

La tercera ejecución introduce el comando de elevación de permisos (sudo java Main):

- **UID y EUID en ejecución normal**: En las primeras pruebas, ambos valores son 501. En macOS, el número 501 es el identificador numérico asignado a la cuenta del primer usuario del sistema (**Heri**).
- **UID y EUID con sudo**: Al anteponer sudo, el sistema solicita la contraseña del usuario y cambia temporalmente la identidad de la ejecución. Tanto el **UID** como el **EUID** pasan a ser 0, que es el identificador reservado para el administrador del sistema (*root*).
- **Cambio de PPID**: El **PPID** cambia a 94651 porque el programa ya no es lanzado directamente por la terminal, sino por el proceso intermedio del comando sudo.

Bibliografía

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.